Randomizing the Montgomery Powering Ladder

Duc-Phong Le[1], Chik How Tan[1], and Michael Tunstall[2](✉)

[1] Temasek Laboratories, National University of Singapore,
5A Engineering Drive 1, #09-02, Singapore 117411, Singapore
{tslld,tsltch}@nus.edu.sg
[2] Rambus Cryptography Research Division,
425 Market Street, 11th Floor, San Francisco, CA 94105, USA
michael.tunstall@cryptography.com

Abstract. In this paper, we present novel randomized techniques to enhance Montgomery powering ladder. The proposed techniques increase the resistance against side-channel attacks and especially recently published correlation collision attacks in the horizontal setting. The first of these operates by randomly changing state such that the difference between registers varies, unpredictably, between two states. The second algorithm takes a random walk, albeit tightly bounded, along the possible addition chains required to compute an exponentiation. We also generalize the Montgomery powering ladder and present randomized (both left-to-right and right-to-left) m-ary exponentiation algorithms.

Keywords: Montgomery powering ladder · Side-channel analysis · Countermeasures

1 Introduction

Side-channel analysis is one of the most serious threats to the security of a given implementation of a cryptographic algorithm. In the traditional model, a given cryptographic algorithm is typically proven secure against various attacks under assumptions regarding the computational complexity of an attack. However, in a more practical scenario, Kocher noted that the time required to compute a cryptographic algorithm could reveal information on the keys used [1]. This was then extended to analyze differences in the power consumption of a microprocessor [2] and the variations in the surrounding electromagnetic field [3,4]. The simplest such attack is based on the inspection of an acquired power consumption (resp. electromagnetic emanation) trace to derive information. This is referred to as Simple Power Analysis (SPA) (resp. Simple ElectroMagnetic Analysis (SEMA)). The exploitation of statistical differences in the instantaneous power consumption proposed by Kocher et al. [2] is termed Differential Power Analysis (DPA) (resp. Differential ElectroMagnetic Analysis (DEMA)), and alternatives have been proposed using, for example, a model and Pearson's correlation coefficient [5] or mutual information [6].

© IFIP International Federation for Information Processing 2015
R.N. Akram and S. Jajodia (Eds.): WISTP 2015, LNCS 9311, pp. 169–184, 2015.
DOI: 10.1007/978-3-319-24018-3_11

In choosing an exponentiation algorithm for a secure implementation, one needs to consider the possible attacks that could be applied. One can often discount attacks where input values need to be known, such as the doubling attack [7], template attacks [8] or DPA [2]. Such attacks can typically be prevented by blinding input values [9] or by using a suitable padding scheme. That is, these attacks are not typically prevented by choosing a specific exponentiation algorithm. However, one also needs to consider attacks based on inspecting a limited number of traces, such as SPA [2], or power attacks in the horizontal setting [10,11]. The later was first introduced by Walter [10]. His attack, the so-called Big Mac attack, applied to m-ary exponentiation, although only simulated attacks are described. Clavier et al. [12] then exploited the collision correlation between selected points from two subtraces to derive information. Recent work by Bauer et al. [13] has also detailed how one could apply such collision attacks to implementations of scalar multiplications over elliptic curves. Witteman et al. [14] demonstrated that this attack works on an ASIC implementation. Kim et al. [15] also determined how one could apply such an attack to the Montgomery ladder [16,17]. The attack model was extended by Hanley et al. [18] to include an attacker that computes the correlation between carefully chosen points in a trace to detect where the output of one operation is used as the input to another operation.

In this paper, we present randomized variants of the Montgomery powering ladder that are resistant to SPA and power collision correlation attacks in the horizontal setting. The first algorithm is based on an amalgamation of two simple variants of the Montgomery powering ladder, where the content of the registers becomes *unpredicable*. The second algorithm is based on blinding addition chains, i.e. it takes tightly bounded random walks to compute an exponentiation. We also generalize the Montgomery powering ladder and present randomized m-ary exponentiation algorithms.

2 Preliminaries

2.1 Attack Model

In this paper we shall predominantly be considering an adversary that is able to take power consumption traces (or something equivalent) while a microprocessor is computing a group exponentiation algorithm. The adversary is then able to make deductions on what the microprocessor is computing. We shall consider three different attacks that require a limited number of traces when discussing the effects of our modifications to the Montgomery powering ladder.

1. The first attack is Simple Power Analysis (SPA) where one observes differences in the power consumption caused by different operations taking place. This was first demonstrated by Kocher *et al.* [2] who showed that one could, given a naïve implementation, observe the difference between operations during the computation of an exponentiation in $(\mathbb{Z}/N\,\mathbb{Z})^*$.

2. Another attack is the use of Pearson's correlation coefficient to detect collisions in variables to deduce key values [11]. For example, during the computation of an exponentiation in $(\mathbb{Z}/N\,\mathbb{Z})^*$ using Coron's *square-and-multiply-always* exponentiation algorithm, one could seek to determine the locations of multiplications with the same input. That is, operations that both take the same input should show a significant cross-correlation.
3. An extension to the collision correlation attack given above is where an adversary is able to detect collisions between the output of one operation and the input of another operation. This provides an attack where a complete defense is not possible. However, limiting the information available to an adversary can make the attack impractical since it has been shown that the error rate for this attack is significantly higher than a straightforward comparison of operations [18].

We do not consider attacks that require chosen inputs, such as the doubling attack [7] or statistical differences in the power consumption over time [2,5]. This is because these attacks are typically prevented by padding or blinding the input by using a redundant representation where the details depend on the group being used [9,19].

Typically, a blinded exponent is used, which is equivalent to the actual exponent, meaning that each trace must be attacked independently. The discussion of the security of the proposed algorithms will be largely informal, except where we wish to make specific claims about the amount of information available to an adversary.

2.2 The Montgomery Powering Ladder

The Montgomery powering ladder was originally proposed as a means of speeding up scalar multiplication over elliptic curves [16], and was later shown to be applicable to all multiplicatively written abelian groups [17].

We recall the description of the Montgomery powering ladder given by Joye and Yen [17]. We consider the problem of computing $y = x^k$ in $\mathbb{G}$ for inputs x and k. Let $\sum_{i=0}^{n-1} k_i\, 2^i$ be the binary expansion of κ with bit length n. For ease of expression we shall also denote this as $(k_{n-1}, \ldots, k_0)_2$. The Montgomery powering ladder relies on the following observation. Defining $L_j = \sum_{i=j}^{n-1} k_i\, 2^{i-j}$ and $H_j = L_j + 1$, we have

$$L_j = 2\,L_{j+1} + k_j = L_{j+1} + H_{j+1} + k_j - 1 = 2\,H_{j+1} + k_j - 2 \tag{1}$$

and so we obtain

$$(L_j, H_j) = \begin{cases} (2\,L_{j+1}, L_{j+1} + H_{j+1}) & \text{if } k_j = 0\,, \\ (L_{j+1} + H_{j+1}, 2\,H_{j+1}) & \text{if } k_j = 1\,. \end{cases} \tag{2}$$

If we consider one register containing x^{L_j} and another containing x^{H_j} then (2) implies that

$$(x^{L_j}, x^{H_j}) = \begin{cases} \left(\left(x^{L_{j+1}}\right)^2, x^{L_{j+1}} \cdot x^{H_{j+1}}\right) & \text{if } k_j = 0\,, \\ \left(x^{L_{j+1}} \cdot x^{H_{j+1}}, \left(x^{H_{j+1}}\right)^2\right) & \text{if } k_j = 1\,. \end{cases}$$

Given that $L_0 = \kappa$ one can build an exponentiation algorithm that requires two group operations per bit of the exponent. Joye and Yen give several different versions of such an algorithm [17]. Algorithm 1 describes the most resistant to side-channel analysis version in their paper (as noted by Kim et al. [15] who describe implementations of cross correlation attacks on other versions).

Algorithm 1. Montgomery Powering Ladder

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, k_{n-2}, \ldots, k_0)_2$
Output: x^κ

```
R_0 ← 1_G ; R_1 ← x ;
for i = n − 1 down to 0 do
    R_{¬k_i} ← R_{k_i} · R_{¬k_i} ;
    R_{k_i} ← (R_{k_i})^2 ;
end
return R_0
```

The Montgomery powering ladder, as described in Algorithm 1, has several properties that make it useful when defining a side-channel resistant implementation of an exponentiation. However, the Montgomery Powering ladder also has been shown to be vulnerable to recent collision correlation attacks in the horizontal setting [12,18]. In the reminder of this paper we propose alternative versions of the Montgomery powering ladder to enhance its security against horizontal collision correlation attacks.

Definition 1. *We define a variant of the Montgomery powering ladder as an exponentiation algorithm that has the following properties.*

1. *The algorithm uses two registers in the main loop containing group elements, both of which are updated in each iteration.*
2. *Each iteration of the main loop treats one bit of the exponent and contains no more than two group operations.*
3. *The operands in the first group operation will only involve one or both of the registers used in the main loop.*
4. *The operands in the second group operation can involve one or both of the registers used in the main loop and/or some precomputed value.*

The 3rd and 4th properties allow for variants of the Montgomery ladder to be defined. That is, the Montgomery ladder is in the set of possible algorithms that satisfy these criteria.

For brevity in defining algorithms, we shall concentrate on the main loop of the algorithm. The computation before and after the main loop may contain **if**-statements. We shall not give fully secure versions where solutions are widely known, e.g. dummy operations or redundant representations [9,19,20].

3 Randomizing the Montgomery Powering Ladder

We note that when computing x^n using the Montgomery powering ladder, as defined in Algorithm 1, then at the end of each iteration we will have the condition where $R_1/R_0 = x$, or equivalently $R_0 \cdot x = R_1$. Thus, using the notation given above, and we allow some precomputed values to be used in the algorithm, then (2) can be rewritten as

$$(L_j, H_j) = \begin{cases} (2\,L_{j+1}, L_j + 1) & \text{if } k_j = 0\,, \\ (L_{j+1} + H_{j+1}, L_j + 1) & \text{if } k_j = 1\,. \end{cases} \tag{3}$$

Implying that

$$(x^{L_j}, x^{H_j}) = \begin{cases} \left(\left(x^{L_{j+1}}\right)^2, x^{L_j} \cdot x\right) & \text{if } k_j = 0\,, \\ \left(x^{L_{j+1}} \cdot x^{H_{j+1}}, x^{L_j} \cdot x\right) & \text{if } k_j = 1\,. \end{cases}$$

From which we can define Algorithm 2.

Algorithm 2. A Straightforward Variant I

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, k_{n-2}, \ldots, k_0)_2$
Output: x^κ

```
R_0 ← 1_G ; R_1 ← x ;
for i = n − 1 down to 0 do
    R_0 ← R_0 · R_{k_i} ;
    R_1 ← R_0 · x ;
end
return R_0
```

Algorithm 3. A Straightforward Variant II

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, k_{n-2}, \ldots, k_0)_2$, and $k_{n-1} = 1$
Output: x^κ

```
R_0 ← 1_G ; R_1 ← 1_G ;
for i = n − 1 down to 0 do
    R_1 ← R_0 · R_{¬k_i} ;
    R_0 ← R_1 · x ;
end
return R_0
```

Following the reasoning used to define (1), we can instead define $L_j = \sum_{i=j}^{n-1} k_i\, 2^{i-j}$ and $H_j = L_j - 1$, giving

$$L_j = 2\,L_{j+1} + k_j = L_{j+1} + H_{j+1} + k_j + 1 = 2\,H_{j+1} + k_j + 2 \tag{4}$$

and so, as with (3), we obtain

$$(L_j, H_j) = \begin{cases} (H_j + 1, L_{j+1} + H_{j+1}) & \text{if } k_j = 0\,, \\ (H_j + 1, 2\,L_{j+1}) & \text{if } k_j = 1\,. \end{cases} \tag{5}$$

If we consider one register containing x^{L_j} and another containing x^{H_j} then (5) implies that

$$(x^{L_j}, x^{H_j}) = \begin{cases} \left(x^{H_j} \cdot x, x^{L_{j+1}} \cdot x^{H_{j+1}}\right) & \text{if } k_j = 0\,, \\ \left(x^{H_j} \cdot x, \left(x^{L_{j+1}}\right)^2\right) & \text{if } k_j = 1\,. \end{cases}$$

From which we can define Algorithm 3. We note that at the end of each iteration we will have the condition where $R_0/R_1 = x$, or equivalently $R_1 \cdot x = R_0$.

Suppose that an adversary can distinguish multiplications from squaring operations, then the two above variants of Montgomery powering ladder are not immune to SPA. In line 3 of both algorithms, a squaring operation will occur for certain bit values. That is, a squaring operation will be computed if the bit value $k_i = 0$ in Algorithm 2 and in Algorithm 3 a squaring operation occurs when $k_i = 1$. The following randomized algorithm will deal with this problem.

We observe that Algorithms 2–3 can be blended together. That is, in loop ℓ, for some $\ell \in \{1, \ldots, n-1\}$, of Algorithm 2 if $k_\ell = 1$ one could compute $R_1 \leftarrow (R_0)^2$ followed by $R_0 \leftarrow R_1 \cdot x$. Before this step $R_1/R_0 = x$, and afterwards one bit of the exponent is treated and $R_0/R_1 = x$. Hence, one could continue to compute an exponentiation using Algorithm 3.

Likewise, in loop ℓ, for some $\ell \in \{1, \ldots, n-1\}$, of Algorithm 3, if $k_\ell = 0$, one could compute $R_0 \leftarrow (R_0)^2$ followed by $R_1 \leftarrow R_0 \cdot x$. Afterwards one bit of the exponent is treated and $R_1/R_0 = x$. Hence, one could continue to compute an exponentiation using Algorithm 2. We define Algorithm 4 as an example of how Algorithms 2 and 3 could be randomly blended together. We use a random generator producing a random bit b to determine when to change from one algorithm to the other, and to determine which algorithm is used to start the exponentiation algorithm. If $b = 0$, the exponentiation is computed by using Algorithm 2, that is $R_1/R_0 = x$. Otherwise, the exponentiation is computed by using Algorithm 3, that is, $R_0/R_1 = x$.

Suppose that an adversary is able to distinguish a multiplication from a squaring operation. Then, she would be able to determine individual bits of the exponent if she could determine if $R_0/R_1 = x$ or $R_1/R_0 = x$. However, the following lemma shows that this information is not available.

Lemma 1. *An adversary analyzing an instance of Algorithm 4 is able to reduce the hypotheses for the exponent from κ to $\kappa^{\frac{11}{12}}$ by distinguishing a multiplication from a squaring operation.*

Proof. If an adversary observes a squaring operation followed by a multiplication in the loop using index i, then the adversary knows one of the following operations has occurred:

Algorithm 4. Randomized Montgomery Powering Ladder

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, k_{n-2}, \ldots, k_0)_2$, and $k_{n-1} = 1$
Output: x^κ

```
b ←R {0,1} ; R_0 ← 1_G ;
if b = 0 then
    R_1 ← x ;
else
    R_1 ← 1_G ;
end
for i = n − 1 down to 0 do
    if b ⊕ k_i = 1 then
        b ←R {0,1} ;
    end
    R_b ← R_0 · R_{b⊕k_i} ;
    R_{¬b} ← R_b · x ;
end
return R_0
```

1. Where $R_1/R_0 = x$ and $k_i = 0$.
2. Where $R_0/R_1 = x$ and $k_i = 1$.
3. Where $R_1/R_0 = x$ changes to $R_0/R_1 = x$ and $k_i = 1$.
4. Where $R_0/R_1 = x$ changes to $R_1/R_0 = x$ and $k_i = 0$.

At an arbitrary point, each of these occur with probability $\frac{1}{4}$. Likewise, if an adversary observes two multiplications in the loop using index i, hence the adversary knows one of the following operations has occurred:

1. Where $R_1/R_0 = x$ and $k_i = 1$.
2. Where $R_0/R_1 = x$ and $k_i = 0$.

Each of these occur with probability $\frac{1}{2}$. Hence, there is no information available to an adversary since for any observed sequence of operations $\Pr(k_i = 0) = \Pr(k_i = 1) = \frac{1}{2}$.

However, if an adversary observes y consecutive pairs of multiplications then the adversary will know that y consecutive bits have the same value. If an attacker observes a pair of multiplications then the distribution of the number of subsequent pairs W of multiplications is geometric. That is, $W \sim \text{Geometric}(\frac{3}{4})$, since the following bit has to be the same and the randomly generated bit has to be a specific value. The, by definition, the expected length of a run of multiplication is $\frac{4}{3}$ operations, where each observation would therefore provide an expected $\frac{4}{3} - 1 = \frac{1}{3}$ of a bit of the exponent. A pair of multiplications will occur with probability $\frac{1}{4}$, giving an expected $n/4$ bits for an n-bit exponent. For an n-bit exponent an expected $\frac{1}{3} \times \frac{n}{4} = \frac{n}{12}$ bits can expected to be derived. Hence, κ hypotheses can be expected to be reduced to $\kappa^{\frac{11}{12}}$ hypotheses. □

As with the Montgomery powering ladder shown in Algorithm 1, an attack using collisions based on the reuse of variables is not possible, but a collision attack based in the use of the output of operations is possible. One can attempt to observe whether the second operand in line 11 of Algorithm 4 is created from line 11 or line 12 in the previous iteration of the exponentiation loop. However, an analysis based on this will return the wrong key hypothesis when the algorithm changes from using (3) to (5).

Lemma 2. *An adversary analyzing an instance of Algorithm 4 using a collision attack an adversary would be able to reduce the hypotheses for the exponent from* κ *to* $\kappa^{\frac{3}{4}}$.

Proof. If we, arbitrarily, consider the ℓ-th loop of the exponentiation loop, we have $\Pr(b \oplus k_i = 1) = \frac{1}{2}$ and the probability that b changes, and hence the algorithm being used, is also $\frac{1}{2}$. An adversary making a deductions using a collision attack will have to guess the value of b in two consecutive loops of the algorithm. Given a correct guess for b it will remain the same with probability $\frac{3}{4}$. On the assumption that an attack will validate b in the first loop the value of b in the second loop will remain the same with probability $\frac{3}{4}$. If it changes an incorrect result will be given. Hence, the probability that a collision would detect an incorrect key bit is $\frac{1}{4}$, and an adversary would be able to determine a given bit with a probability $\frac{3}{4}$ leading to a reduction in the number of hypotheses from κ to $\kappa^{\frac{3}{4}}$. □

4 Random Walk Method

In this section, we generalize the difference between the two registers used in the Montgomery powering ladder to be some arbitrary power of the input. This leads to an algorithm that computes a group exponentiation taking a random, albeit tightly bounded, walk through the possible addition chains.

If one is working in a group where computing the inverses of an element can be readily computed, then other options for a variant of the Montgomery powering ladder are possible. We note that (2) can be rewritten as

$$(L_j, H_j) = \begin{cases} (H_j - 1, L_{j+1} + H_{j+1}) & \text{if } k_j = 0\,, \\ (L_{j+1} + H_{j+1}, L_j + 1) & \text{if } k_j = 1\,. \end{cases} \tag{6}$$

Implying that

$$(x^{L_j}, x^{H_j}) = \begin{cases} \left(x^{H_j} \cdot x^{-1}, x^{L_{j+1}} \cdot x^{H_{j+1}}\right) & \text{if } k_j = 0\,, \\ \left(x^{L_{j+1}} \cdot x^{H_{j+1}}, x^{L_j} \cdot x\right) & \text{if } k_j = 1\,. \end{cases}$$

(6) can be rewritten as follows:

$$(L_j, H_j) = \begin{cases} (L_{j+1} + H_{j+1}, L_j - 1) & \text{if } k_j = 0\,, \\ (L_{j+1} + H_{j+1}, L_j + 1) & \text{if } k_j = 1\,. \end{cases} \tag{7}$$

Implying that

$$(x^{L_j}, x^{H_j}) = \begin{cases} \left(x^{L_{j+1}} \cdot x^{H_{j+1}}, x^{L_j} \cdot x^{-1}\right) & \text{if } k_j = 0\,, \\ \left(x^{L_{j+1}} \cdot x^{H_{j+1}}, x^{L_j} \cdot x\right) & \text{if } k_j = 1\,. \end{cases}$$

From which we can define Algorithm 5. In previous examples of Montgomery powering ladders presented in this paper R_0 has acted as an accumulator and returned the result. In Algorithm 5 the accumulator, i.e. the register containing the correct power of x at the end of each loop, shifts depending on the value of the bit of the exponent being treated. Hence, $R_{\neg k_0}$ is returned at the end of the algorithm.

Algorithm 5. Variant with Inverses

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, k_{n-2}, \ldots, k_0)_2$
Output: x^κ

1 $R_0 \leftarrow 1_{\mathbb{G}}$; $R_1 \leftarrow x$;
2 $U_0 \leftarrow x^{-1}$; $U_1 \leftarrow x$;
3 **for** $i = n-1$ **down to** 0 **do**
4 $\quad R_0 \leftarrow R_0 \cdot R_1$;
5 $\quad R_1 \leftarrow R_0 \cdot U_{k_i}$;
6 **end**
7 **return** $R_{\neg k_0}$

In analyzing an instance of Algorithm 5, an adversary would not be able to determine any information on bits of the exponent based on distinguishing a multiplication from a squaring operation, since no squaring operations take place. However, a collision attack is possible by observing where the multiplication with U_i, for $i \in \{0, 1\}$, collides with the multiplications used to generate these values in line 2. If an adversary is able to determine whether $U_{k_i} = x^{-1}$ or $U_{k_i} = x$ was used, for some $i \in \{0, \ldots, n-1\}$, then individual bits of the exponent can be determined.

We can modify (7) by choosing $L'_{j+1} = L_{j+1} + \alpha$ and $H'_{j+1} = H_{j+1} + \beta$, giving

$$(L'_j, H'_j) = \begin{cases} (L'_{j+1} + H'_{j+1}, L'_j - 1) & \text{if } k_j = 0\,, \\ (L'_{j+1} + H'_{j+1}, L'_j + 1) & \text{if } k_j = 1\,. \end{cases} \tag{8}$$

where $L'_j = L_j + \gamma$, $H'_j = H_j + \mu$, and $\gamma = \alpha + \beta$. If we choose γ as a random element from $\{-h, \ldots h\}$ for some small integer h, then α and β can be chosen such that $\gamma = \alpha + \beta$. If we assume that α is fixed then $\beta = \gamma - \alpha$, i.e., $H'_{j+1} = H_{j+1} + \gamma - \alpha$. Given that H'_{j+1} is computed from L'_{j+1} this can be done by

$$H'_{j+1} = \begin{cases} L_{j+1} + (\gamma - \alpha - 1) = L'_{j+1} + (\gamma - 2\,\alpha - 1) & \text{if } k_j = 0\,, \\ L_{j+1} + (\gamma - \alpha + 1) = L'_{j+1} + (\gamma - 2\,\alpha + 1) & \text{if } k_j = 1\,. \end{cases}$$

This also removes the need to have the accumulating register change as described for Algorithm 5. If we define a value $L'_j = L_j + \gamma_j$ then (8) can be rewritten as

$$(L_j + \gamma_j, H_j + (\gamma_{j-1} - \gamma_j) =$$

$$\begin{cases} \big(L_{j+1} + \gamma_{j+1} + H_{j+1} + (\gamma_j - \gamma_{j+1}), L'_j + (\gamma_{j-1} - 2\,\gamma_j - 1)\big) & \text{if } k_j = 0\,, \\ \big(L_{j+1} + \gamma_{j+1} + H_{j+1} + (\gamma_j - \gamma_{j+1}), L'_j + (\gamma_{j-1} - 2\,\gamma_j + 1)\big) & \text{if } k_j = 1\,. \end{cases} \tag{9}$$

If we consider one register containing $x^{L'_j}$ and another containing $x^{H'_j}$ then (9) implies that $(x^{L'_j}, x^{H'_j}) = \left(x^{L'_{j+1}} \cdot x^{H'_{j+1}}, x^{L'_j} \cdot x^{\Delta}\right)$ where

$$\Delta = \begin{cases} \gamma_{j-1} - 2\,\gamma_j - 1 & \text{if } k_j = 0\,, \\ \gamma_{j-1} - 2\,\gamma_j + 1 & \text{if } k_j = 1\,. \end{cases}$$

Given that $L_0 = \kappa$ one can build an exponentiation algorithm as shown in Algorithm 6 where we set $\gamma_0 = 0$ to produce the correct result. Assume that γ can be arbitrarily chosen from the set $\{-h, \ldots, h\}$ in each iteration of computation, then $\Delta \in \{-3\,h - 1, \ldots, 3\,h + 1\}$. Hence, our algorithm makes use of an array of $6\,h + 3$ elements that stores the required values of x^{Δ}.

Algorithm 6. Blinded Montgomery Powering Ladder

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, \ldots, k_0)_2$, small integer $h \in \mathbb{Z}$
Output: x^{κ}
Uses: U a $6\,h + 3$ element array.

```
U_{3h+1} ← 1_G ;
for i = 1 to 3h + 1 do
    U_{3h+1+i} ← U_{3h+i} · x ;
    U_{3h+1−i} ← U_{3h+2−i} · x^{−1} ;
end
R_0 ← 1_G ; R_1 ← 1_G ; α = 0 ;
for i = n − 1 down to 1 do
    R_0 ← R_0 · R_1 ;
    γ ←^R {−h, ..., h} ;
    ℓ ← γ − 2α + k_i − ¬k_i ;
    R_1 ← R_0 · U_{3h+1+ℓ} ;
    α ← γ ;
end
R_0 ← R_0 · R_1 ;
R_0 ← R_0 · U_{3h+1−α−¬k_0} ;
return R_0
```

An adversary who is able to distinguish a multiplication from a squaring operation would not be able to determine any information on bits of the exponent used since no squaring operations take place. However, a cross correlation attack is possible by observing where the multiplication with U_d, for $d \in \{0, \ldots, 6\,h+2\}$,

collides with the multiplications used to generate these values in line 11. From these values, an adversary can derive the addition chain that was used to compute the exponentiation. This will not give an adversary the exponent, since there is no means to map the digits used to bits of the exponent, but would give an adversary an equivalent addition chain.

In determining whether an attack is practical Hanley et al. [18] determine that when analyzing an implementation 192-bit exponentiation, the error rate need to be less than 22 bits. This is determined to be less than 2^{54} operations, based on the boundary set for block ciphers by Biryukov et al. [21]. The expected number of operations can be determined using Stinson's algorithm [22], where an t-bit error in a n-bit hypothesis leads to the exponent in time complexity $\mathcal{O}\left(n \sum_{i=0}^{\lceil t/2 \rceil} \binom{n/2}{i}\right)$ [22]. Then one can apply a version of Stinson's algorithm where one has to treat all the values that digits can take. Given a small integer h, as defined in Algorithm 6, a t-digit error in an μ-bit hypothesis leads to the exponent in time complexity $\mathcal{O}\left(\mu \binom{\mu/2}{t/2} (2\,h+1)^{\frac{\mu}{2}}\right)$. That is, there are μ possible divisions of the digits, each of which will have $\binom{\mu/2}{t/2}$ ways of selecting $t/2$ digits and each digit can take $(2\,h+1)$ values. In practice, t will not be known so the analysis will have time complexity $\mathcal{O}\left(\mu \sum_{i=0}^{\lceil t/2 \rceil} \binom{\mu/2}{i} (2\,h+1)^i\right)$ further reducing the required error rate for a successful attack.

We note that an adversary is required to derive the entire exponent as information on part of the exponent is not useful. Moreover, an adversary will not know the exponent but an equivalent addition-chain. If we consider the group exponentiation algorithm used in ECDSA (Elliptic Curve Digital Signature Algorithm) [23] this provides a significant increase in security. Howgrave-Graham and Smart [24] noted that if a few bits of the ephemeral exponent are known for sufficiently many signatures, then the scheme can be broken, based on the so-called hidden number problem introduced by Boneh and Venkatesan [25].

A more memory-efficient version of Algorithm 6 is described in Algorithm 7. This is a security-memory trade off where the value of γ_j is still chosen at random, but the value range depend on the previous value γ_{j+1}. Although the available choices for γ_j decreases, Algorithm 7 requires only $2\,h+2$ registers to store pre-computed values instead of $6\,h+3$ registers in Algorithm 6.

5 Generalizing the Montgomery Powering Ladder

The *square-and-multiply* (left-to-right and right-to-left)) exponentiation algorithms are the most efficient implementations for raising x to the power κ when the exponent is treated bit-by-bit. Furthermore, these exponentiation algorithms extend easily to any radix m for the purpose of speeding the computation. In this section, by using the random walk technique, we generalize the (both left-to-right and right-to-left) exponentiation algorithms and present new blinded m-ary exponentiation algorithms.

Algorithm 7. Blinded Montgomery Powering Ladder II

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (k_{n-1}, k_{n-2}, \ldots, k_0)_2$, small integer $h \in \mathbb{Z}$, U a look-up table of $2h+2$ precomputed values $x^{-h}, \ldots, x^{h+1}$.
Output: x^κ

```
U_h ← 1_G ;
for i = 1 to h do
    U_{h+i} ← U_{h+i-1} · x ;
    U_{h-i} ← U_{h+1-i} · x^{-1} ;
end
U_{2h+1} ← U_{2h} · x ; R_0 ← 1_G ; R_1 ← 1_G ; α ← 0 ;
for i = n − 1 down to 1 do
    R_0 ← R_0 · R_1 ;
    if α ≥ 0 then
        γ ←^R {2α − h, ..., h + 1 − k_i} ;
    end
    else
        γ ←^R {−h − k_i, ..., 2α + h − 1};
    end
    R_1 ← R_0 · U_{h+γ+k_i};
    α ← 2α − γ ;
end
R_0 ← R_0 · R_1 ;
R_0 ← R_0 · U_{h+α+k_0} ;
return R_0
```

5.1 Left-to-Right Algorithms

We first consider the left-to-right exponentiation. Let $\kappa = (w_{\ell-1}, w_{\ell-2}, \ldots, w_0)_2$ be the m-ary representation of κ, where $0 \neq w_i < m$, $w_{\ell-1} \neq 0$, and ℓ is the length of κ in radix m. As in Section 2.2, by defining $L_j = \sum_{i=j}^{\ell-1} w_i\, m^{i-j}$, we have

$$L_j = m\, L_{j+1} + w_j \tag{10}$$

From then, the exponentiation algorithm will perform $x^{L_j} = (x^{L_{j+1}})^m \cdot x^{w_j}$. Noting that $x^{L_0} = x^\kappa$. Using the random walk technique as in Section 4, we modify $L'_j = L_j + \gamma_j$ where γ_j can be chosen from a pre-defined set $\{-h, \ldots, h\}$ for some small integer h. Then (10) can be rewritten as

$$L_j + \gamma_j = m\, L_{j+1} + \gamma_j + w_j = m\, L'_{j+1} - m\, \gamma_{j+1} + \gamma_j + w_j \tag{11}$$

This leads to a randomized algorithm (Algorithm 8) that computes a group exponentiation taking a random, albeit tightly bounded, walk through the possible addition chains. As with Algorithm 7, Algorithm 8 also requires a look-up table of $2(m+1)\,h + m - 1$ values to stores precomputated values. Algorithm 8

can be used in the cases where the computation of inversions is cheap, otherwise Algorithm 9 is more suitable.

Algorithm 8. Randomized Left-to-Right m-ary Exponentiation

Input: $x \in \mathbb{G}$, an integer $\kappa = (w_{\ell-1}, w_{\ell-2}, \ldots, w_0)_m$, where $0 \leq w_i < m$ and $w_{\ell-1} \neq 0$, small integer $h \in \mathbb{Z}$, T a look-up table of $2(m+1)\,h + m - 1$ precomputed values $x^{-(m+1)h-m+1}, \ldots, x^{(m+1)h+m-1}$.

Output: x^κ

$R \leftarrow 1_{\mathbb{G}}$, $\alpha \leftarrow 0$
for $i = \ell - 2$ **to** 1 **do**
  $R \leftarrow R^m$;
  $\gamma \xleftarrow{R} \{-h, \ldots, h\}$;
  $R \leftarrow R \cdot x^{m\,\alpha - \gamma + w_i}$;
  $\alpha \leftarrow \gamma$;
end
$R \leftarrow R^m$;
$R \leftarrow R \cdot x^{m\,\alpha + w_0}$;
return R

Algorithm 9. Randomized Left-to-Right m-ary Exponentiation without Inversions

Input: $x \in \mathbb{G}$, an integer $\kappa = (w_{n-1}, w_{n-2}, \ldots, w_0)_m$, where w_i is m-bit words, small integer $h \in \mathbb{Z}$, T a look-up table of $m\,(h+1) - 1$ precomputed values $x, \ldots, x^{m(h+1)-1}$.

Output: x^κ

$R \leftarrow 1_{\mathbb{G}}$, $\alpha \leftarrow 0$
for $i = \ell - 2$ **to** 1 **do**
  $R \leftarrow R^m$;
  $\gamma \xleftarrow{R} \{0, \ldots, \min(h, m\,\alpha + w_i)\}$;
  $R \leftarrow R \cdot x^{m\,\alpha - \gamma + w_i}$;
  $\alpha \leftarrow \gamma$;
end
$R \leftarrow R^m$;
$R \leftarrow R \cdot x^{m\,\alpha + w_0}$;
return R

We note that Algorithm 9 works in a similar way to the Overlapping Windows method [26] for a fixed base $m = 2^{k-h_{OWM}}$ and $m(h+1) = 2^k$[1]. The main difference is that our algorithm generates *on-the-fly* a randomized recoding of the binary representation of the secret exponent κ. This allows our algorithm to avoid side-channel attacks in the recoding phase.

5.2 Right-to-Left Algorithm

Likewise, we can devise a randomized *right-to-left* m-ary algorithm. Let $\kappa = (w_{\ell-1}, w_{\ell-2}, \ldots, w_0)_m$, where $w_{\ell-1} \neq 0$, the principle of the *right-to-left* m-ary exponentiation algorithm, as shown by Yao [27], makes use of the is relied on the following equality:

$$x^\kappa = \prod_{\substack{0 \leq i \leq \ell-1 \\ d_i = 1}} x^{m^i} \cdot \prod_{\substack{0 \leq i \leq \ell-1 \\ d_i = 2}} x^{2 \cdot m^i} \cdots \prod_{\substack{0 \leq i \leq \ell-1 \\ d_i = m-1}} x^{(m-1) \cdot m^i} = \prod_{j=1}^{m-1} \Big(\prod_{\substack{0 \leq i \leq \ell-1 \\ d_i = j}} x^{m^i} \Big)^j .$$

[1] We use the notation h_{OWM} instead of h as in [26] to distinguish it from the notation h in our algorithms.

The *right-to-left* m-ary exponentiation algorithm uses $(m-1)$ accumulators, $R[1], \ldots, R[m-1]$. At each iteration, it applies w successive squarings to compute $A = x^{m^j}$ from $x^{m^{(j-1)}}$, then multiplies the result to some accumulators $R[k]$. Let $R[k]^{(j)}$ (resp. $A^{(j)}$) denote the value of the accumulator $R[k]$ (resp. A) before entering step j. We have:

$$\begin{aligned} R[k]^{(j+1)} &= R[k]^{(j)} \cdot A^{(j)} \quad && \text{for } k = w_j\,, \\ R[k]^{(j+1)} &= R[k]^{(j)} && \text{for } k \neq w_j\,, \end{aligned}$$

and $A^{(j+1)} = (A^{(j)})^m$. At the end of the loop each accumulator $R[k]$ contains the product $\prod_{\substack{0 \le j \le \ell-1 \\ w_j = k}} x^{m^j}$. The different accumulators are finally aggregated as $\prod_{0 \le j \le \ell-1} R[k]^k = x^\kappa$. By defining $L_j = \sum_{i=0}^{j} w_i \cdot m^i$, we have

$$L_{j+1} = m^{j+1} \cdot w_{j+1} + Lj \tag{12}$$

Noting that $L_{\ell-1} = \kappa$. Similar, we use the random walk technique and modify $L'_j = L_j - m^{j+1} \cdot \gamma_j$, then equation (12) can be rewritten as:

$$\begin{aligned} L_{j+1} - m^{j+2} \cdot \gamma_{j+1} &= m^{j+1} \cdot w_{j+1} + L_j - m^{j+2} \cdot \gamma_{j+1} \\ &= m \cdot w_{j+1} + L'_j + m^{j+1} \cdot \gamma_j - m^{j+2} \cdot \gamma_{j+1} \\ &= (\gamma_j - m \cdot \gamma_{j+1} + w_{j+1}) m^{j+1} + L'_j \end{aligned}$$

This leads to a randomized *right-to-left* m-ary exponentiation algorithm (Algorithm 10). In this algorithm, we make use of $2(m+1)\,h + m - 1$ accumulators $R[j]$, where $-(m+1)h - m + 1 \le j \le (m+1)h + m - 1$. Each $R[j]$ is initialized to $1_{\mathbb{G}}$, and then updated if $\alpha - m\gamma + w_i = j$. The different accumulators are finally aggregated as $\prod_{j=1}^{h} \left(R[j] \cdot (R[-j])^{-1}\right)^j = x^\kappa$.

6 Conclusion

In this paper we presented variants of the Montgomery powering ladder which have properties that increase the side-channel resistance of the exponentiation algorithm. The first of these operates by randomly changing states such that the difference between the two registers varies, unpredictably, between two states. The second variant of the Montgomery powering ladder that we presented takes a random walk, albeit tightly bounded, among the possible addition chains required to compute an exponentiation. While this variant is not resistant to all side-channel analysis, it will prevent lattice-based attacks when used, for example, in implementations of ECDSA. In other cases, significantly more computation is required to derive any exploitable information and, therefore, an adversary requires a lower error rate to succeed. By applying the random walk method, we also generalized the Montgomery powering ladder and present randomized (both left-to-right and right-to-left) m-ary exponentiation algorithms.

Algorithm 10. Randomized Right-to-Left m-ary Exponentiation

Input: $x \in \mathbb{G}$, an n-bit integer $\kappa = (w_{\ell-1}, w_{\ell-2}, \ldots, w_0)_m$, small integer $h \in \mathbb{Z}$.
Output: x^κ

```
for j = 0 to (m+1)h + m − 1 do
    R[j] ← 1_G ; R[−j] ← 1_G ;
end
A ← x ; α ← 0 ;
for i = 0 to ℓ − 1 do
    γ ←^R {−h, ..., h} ;
    R[α − mγ + k_i] ← R[α − mγ + k_i] · A ;
    α ← γ ; A ← A^2 ;
end
A ← ∏_{j=1}^{h} (R[j] · (R[−j])^{−1})^j ;
return A
```

References

1. Kocher, P.C.: Timing attacks on implementations of diffie-hellman, RSA, DSS, and other systems. In: Koblitz, N. (ed.) CRYPTO 1996. LNCS, vol. 1109, pp. 104–113. Springer, Heidelberg (1996)
2. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M. (ed.) CRYPTO 1999. LNCS, vol. 1666, p. 388. Springer, Heidelberg (1999)
3. Quisquater, J.-J., Samyde, D.: ElectroMagnetic analysis (EMA): measures and counter-measures for smart cards. In: Attali, S., Jensen, T. (eds.) E-smart 2001. LNCS, vol. 2140, p. 200. Springer, Heidelberg (2001)
4. Gandolfi, K., Mourtel, C., Olivier, F.: Electromagnetic analysis: concrete results. In: Koç, Ç.K., Naccache, D., Paar, C. (eds.) CHES 2001. LNCS, vol. 2162, p. 251. Springer, Heidelberg (2001)
5. Brier, E., Clavier, C., Olivier, F.: Correlation power analysis with a leakage model. In: Joye, M., Quisquater, J.-J. (eds.) CHES 2004. LNCS, vol. 3156, pp. 16–29. Springer, Heidelberg (2004)
6. Gierlichs, B., Batina, L., Tuyls, P., Preneel, B.: Mutual information analysis. In: Oswald, E., Rohatgi, P. (eds.) CHES 2008. LNCS, vol. 5154, pp. 426–442. Springer, Heidelberg (2008)
7. Fouque, P.-A., Valette, F.: The doubling attack – *why upwards is better than downwards*. In: Walter, C.D., Koç, Ç.K., Paar, C. (eds.) CHES 2003. LNCS, vol. 2779, pp. 269–280. Springer, Heidelberg (2003)
8. Chari, S., Rao, J.R., Rohatgi, P.: Template attacks. In: Kaliski Jr., B.S., Koç, Ç.K., Paar, C. (eds.) CHES 2002. LNCS, vol. 2523, pp. 13–28. Springer, Heidelberg (2003)
9. Coron, J.-S.: Resistance against differential power analysis for elliptic curve cryptosystems. In: Koç, Ç.K., Paar, C. (eds.) CHES 1999. LNCS, vol. 1717, p. 292. Springer, Heidelberg (1999)
10. Walter, C.D.: Sliding windows succumbs to big MAC attack. In: Koç, Ç.K., Naccache, D., Paar, C. (eds.) CHES 2001. LNCS, vol. 2162, p. 286. Springer, Heidelberg (2001)

11. Clavier, C., Feix, B., Gagnerot, G., Giraud, C., Roussellet, M., Verneuil, V.: ROSETTA for single trace analysis. In: Galbraith, S., Nandi, M. (eds.) INDOCRYPT 2012. LNCS, vol. 7668, pp. 140–155. Springer, Heidelberg (2012)
12. Clavier, C., Feix, B., Gagnerot, G., Roussellet, M., Verneuil, V.: Horizontal correlation analysis on exponentiation. In: Soriano, M., Qing, S., López, J. (eds.) ICICS 2010. LNCS, vol. 6476, pp. 46–61. Springer, Heidelberg (2010)
13. Bauer, A., Jaulmes, E., Prouff, E., Wild, J.: Horizontal collision correlation attack on elliptic curves. In: Lange, T., Lauter, K., Lisoněk, P. (eds.) SAC 2013. LNCS, vol. 8282, pp. 553–570. Springer, Heidelberg (2014)
14. Witteman, M.F., van Woudenberg, J.G.J., Menarini, F.: Defeating RSA multiply-always and message blinding countermeasures. In: Kiayias, A. (ed.) CT-RSA 2011. LNCS, vol. 6558, pp. 77–88. Springer, Heidelberg (2011)
15. Kim, H., Kim, T.H., Yoon, J.C., Hong, S.: Practical second-order correlation power analysis on the message blinding method and its novel countermeasure for RSA. ETRI J. **32**, 102–111 (2010)
16. Montgomery, P.: Speeding the Pollard and elliptic curve methods of factorization. Mathematics of Computation **48**, 243–264 (1987)
17. Joye, M., Yen, S.-M.: The montgomery powering ladder. In: Kaliski Jr., B.S., Koç, Ç.K., Paar, C. (eds.) CHES 2002. LNCS, vol. 2523, pp. 291–302. Springer, Heidelberg (2003)
18. Hanley, N., Kim, H.S., Tunstall, M.: Exploiting collisions in addition chain-based exponentiation algorithms using a single trace. In: Nyberg, K. (ed.) CT-RSA 2015. LNCS, vol. 9048, pp. 429–446. Springer, Heidelberg (2015)
19. De Win, E., Mister, S., Preneel, B., Wiener, M.: On the performance of signature schemes based on elliptic curves. In: Buhler, J.P. (ed.) ANTS 1998. LNCS, vol. 1423, pp. 252–266. Springer, Heidelberg (1998)
20. Mangard, S., Oswald, E., Popp, T.: Power Analysis Attacks – Revealing the Secrets of Smart Cards. Springer (2007)
21. Biryukov, A., Dunkelman, O., Keller, N., Khovratovich, D., Shamir, A.: Key recovery attacks of practical complexity on AES-256 variants with up to 10 rounds. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 299–319. Springer, Heidelberg (2010)
22. Stinson, D.: Some baby-step giant-step algorithms for the low Hamming weight discrete logarithm problem. Mathematics of Computation **71**, 379–391 (2002)
23. X9.62, A.: Public key cryptography for the financial services industry, the elliptic curve digital signature algorithm (ECDSA) (1999)
24. Howgrave-Graham, N.A., Smart, N.P.: Lattice attacks on digital signature schemes. Design, Codes and Cryptography **23**, 283–290 (2001)
25. Boneh, D., Venkatesan, R.: Hardness of computing the most significant bits of secret keys in diffie-hellman and related schemes. In: Koblitz, N. (ed.) CRYPTO 1996. LNCS, vol. 1109, pp. 129–142. Springer, Heidelberg (1996)
26. Itoh, K., Yajima, J., Takenaka, M., Torii, N.: DPA Countermeasures by Improving the window method. In: Kaliski Jr., B.S., Koç, Ç.K., Paar, C. (eds.) CHES 2002. LNCS, vol. 2523, pp. 303–317. Springer, Heidelberg (2003)
27. Yao, A.C.C.: On the evaluation of powers. SIAM J. Comput. **5**, 100–103 (1976)